IDs: 213851579, 213310485

Video Processing – Final Project

Report

Course 0512.4263 2025/2026

Overview

The project processes a shaky video of a walking person in four stages. Each stage consumes the previous stage's output:

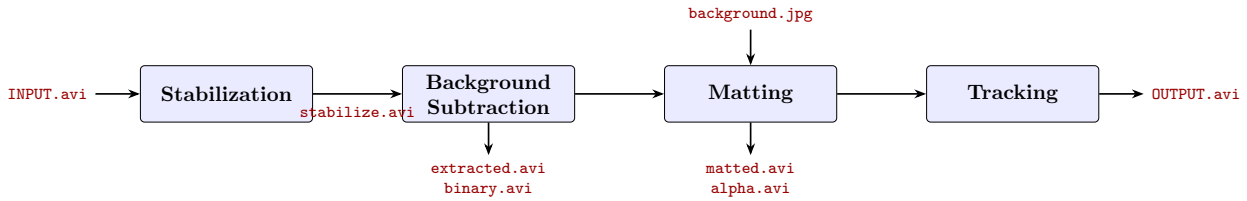

Figure 1: The processing pipeline. Blue blocks are the implemented stages; red names are the video/image artifacts each stage reads or writes.

1 Stabilization

Input. `INPUT.avi` – a shaky, handheld color video of a person walking.

Expected output. `stabilize_ID1_ID2.avi` – the same video, in color, with the camera motion cancelled: every frame is warped onto the coordinate system of the first frame, so the background stays fixed for the next stage. Black regions near the frame borders are a known side effect of the warp and are allowed.

Algorithm. We detect Shi–Tomasi corners and track them with pyramidal Lucas–Kanade optical flow, both exactly as taught in the course (and implemented in Exercise 2; here we call OpenCV's implementation for speed). A robust fit with RANSAC, also as seen in the lectures, maps the tracked corners' positions onto a single global motion model per frame, and rejects the corners that sit on the walking person, whose motion disagrees with the camera's. The frame is then warped once, in color, by the fitted transform.

Two choices differ from the straightforward recipe and need a word:

- *Anchored fit instead of composed transforms.* The usual approach estimates the motion from frame t to $t-1$ and composes all these transforms to reach frame 0. Composing ∼200 noisy estimates accumulates error, and the background slowly drifts. Instead, every tracked corner

carries its frame-0 position (its "anchor"), and each frame we fit the transform *directly* from current positions to anchors. With no composition chain, drift cannot build up. Corners lost to occlusion or RANSAC rejection are replaced by freshly detected ones, anchored through the current fit.

- *Homography as the motion model.* From the transformation ladder seen in class (translation $\rightarrow$ affine $\rightarrow$ homography) we use the homography. A handheld camera also rotates slightly out of the image plane, which creates a small perspective (keystone) distortion between frames; similarity and affine models cannot represent it, and with them a residual wobble remains visible.

Result. In the output video the scene is static: walls, doors and posters stay in place through the entire clip and only the person moves. Averaging all output frames gives a sharp image of the corridor with a faint ghost of the walking person, which is exactly the behaviour a stabilized static background should show. Narrow black wedges appear at the frame borders, as expected.

2 Background Subtraction

Input. `stabilize_ID1_ID2.avi` – the stabilized color video from stage 1.

Expected output. Two videos: `extracted_ID1_ID2.avi`, where the person keeps the original pixel values and everything else is black, and `binary_ID1_ID2.avi`, the matching mask (person = 1, saved scaled to 255).

Algorithm. The core is the per-pixel Gaussian Mixture Model background model from the lectures, used as taught: each pixel holds $K = 4$ Gaussians $(\mu_k, \sigma_k, \omega_k)$, a new sample matches a Gaussian within 2.5σ, matched Gaussians are updated and their weight grows, unmatched ones decay, the weakest is replaced on no match, and the background Gaussians are the top ones by ω_k/σ_k up to a cumulative weight $T = 0.7$, inside the lecture's own suggested range of 0.7–0.9. The departures from the lecture version:

- *Offline instead of online.* The lecture's algorithm serves a live camera. We have the full video on disk, so the model is trained on the whole clip first and only then is every frame classified, against the final model.
- *Median/MAD initialization.* The main Gaussian of every pixel starts at the pixel's temporal median color, with σ from the median absolute deviation. The person covers any single pixel for a minority of the frames, so the median already *is* the background; starting there instead of from a blind guess avoids pixels that never converge within the clip's length.
- *CIE-Lab color space.* Matching runs in Lab with the illumination channel down-weighted: $d^2 = 0.25\,\Delta L^2 + \Delta a^2 + \Delta b^2$. Shadows and lighting changes live almost entirely in L, while the person differs in chroma, so this makes the classifier insensitive to illumination but sharp on real color change. A pixel whose L drops to 50–96% of the background's while a, b stay within 8 units is labelled a cast shadow and kept as background.
- *Cleanup.* Morphological opening and closing (recommended in the lecture notes), keeping every connected component at least 20% of the largest one (dark clothing can split a limb off the torso; keeping only the largest deletes real body parts), hole filling, and a smooth upscale of the mask back to full resolution.

Result. The binary mask holds the complete silhouette – head, arms, legs, shoes – in all frames, with no background patches, and the person's cast shadow on the floor is not included. The extracted video shows the person cleanly cut out on black, with a smooth boundary, ready for matting.

3 Matting

Input. Three artifacts: `stabilize_ID1_ID2.avi` (the stabilized color video), `binary_ID1_ID2.avi` (the person mask from stage 2), and `background.jpg` (the new background image, resized to the video resolution).

Expected output. Two videos: `matted_ID1_ID2.avi`, the person composited onto the new background, and `alpha_ID1_ID2.avi`, the per-pixel opacity $\alpha \in [0, 1]$ stored scaled to $[0, 255]$ and replicated to three channels.

Algorithm. For every frame we run the matting pipeline from Lecture 10: build a *trimap*, learn a *color model (KDE)*, find the *opacity* α from geodesic distances, and *composite* the person onto the new background. To stay fast, we only work inside a small box around the person (found from the mask); the rest of the frame is just background, so we copy the new background there directly. The steps:

1. *Trimap.* We erode and dilate the mask with a disc of radius $\rho = 7$. The eroded part is surely the person ($\alpha = 1$), everything outside the dilated part is surely background ($\alpha = 0$), and the thin ring in between is the "unknown" band, where we still have to find α.
2. *Color model.* Using the pixels next to the band, we learn two color models: how likely a color belongs to the person, and how likely it belongs to the background. A KDE (a smoothed color histogram) gives these two likelihoods, and Bayes' rule turns them into the probability that a pixel is foreground, $P(F \mid c) = \dfrac{f(c \mid F)}{f(c \mid F) + f(c \mid B)}$.
3. *Opacity.* We connect the band pixels in a small graph, where the cost of moving between two neighbours is how much their foreground probability differs. The shortest distance from a pixel to the sure-foreground and sure-background pixels (D_F and D_B) is found with Dijkstra. Alpha then mixes distance and color probability:

   $$w_F = D_F^{-r}\, P(F \mid c), \qquad w_B = D_B^{-r}\, P(B \mid c), \qquad \alpha = \frac{w_F}{w_F + w_B}, \qquad r = 2.$$

   A pixel that is close to the person and person-colored gets α near 1, and the opposite gets α near 0.
4. *Compositing.* Just blending the band pixel's own color would mix in the old background. Instead, for each band pixel we look at the 7 nearest sure-foreground and 7 nearest sure-background pixels, try every pair, and keep the pair whose blend looks most like the real pixel. We then use that clean foreground color in the final blend, $J = \alpha\, c(x_F^*) + (1 - \alpha)\, B_{\text{new}}$.

What we changed or added on top of the lecture:

- *Trimap from the mask.* In class the trimap comes from a few hand-drawn scribbles. We already have a full mask from stage 2, so we just erode and dilate it to get the trimap – no drawing and no user input.
- *Faster KDE.* Computing the KDE the direct way (a Gaussian around every sample, for every pixel) is very slow. Instead we put the colors into a 64^3 histogram and blur it once with a Gaussian; after that, looking up a pixel's color is instant. This is what keeps the matting fast.
- *Geodesic distance with a graph.* We turn the "distance" idea from class into a shortest-path problem on a pixel graph and solve it with Dijkstra.
- *Nearest pixels with a KD-tree.* To find the closest sure pixels quickly, we use a KD-tree instead of checking every pixel one by one.

None of these library tools does the matting for us: Dijkstra, the KD-tree and the Gaussian blur are just small helpers, and all the real decisions are written by us. We use no deep learning, and we do

not use the OpenCV-contrib matting functions (which would cap the grade at 70 points). The whole per-frame work is done in `process_frame`, helped by `kde_3d` (the fast KDE) and `pure_composite` (the clean-color blend).

Result. The alpha video is white on the person and black on the background, with a thin grey edge along the outline. The matted video shows the person on the new background with clean edges and no leftover color from the old corridor, so the clean-color step works. Since we only work inside the box around the person and the KDE is fast, this stage runs well within the time limit.

4 Tracking

Input. The composited video `matted_ID1_ID2.avi` from stage 3. The mask video `binary_ID1_ID2.avi` is read only, to initialize the tracker automatically.

Expected output. `OUTPUT_ID1_ID2.avi`, the matted video with a green rectangle drawn around the tracked person, and `tracking.json`, mapping each frame "1"… "N" to the box `[ROW, COL, HEIGHT, WIDTH]` (top-left corner).

Algorithm. We track the person with the particle filter (Condensation) from the tracking lecture, the same one we built in Exercise 3. Each particle is a guess for the box, written as $s = [x_c, y_c, w_{1/2}, h_{1/2}, v_x, v_y]$ (center, half-size, and velocity). We keep $N = 100$ particles and update them every frame in four steps:

1. *Sample.* Pick N particles from the previous ones, giving heavier (better) particles a higher chance of being picked. This is done by drawing a random number $r \in [0, 1]$ and reading off the first particle whose running weight sum (the CDF) reaches r.
2. *Predict.* Move each particle by its velocity, then add a little random noise to its position and velocity, so the particles spread out and can follow the person.
3. *Measure.* For each particle's box we build a color histogram ($16 \times 16 \times 16$ bins) and compare it to the person's reference histogram q with the Bhattacharyya coefficient $\mathrm{BC}(p, q) = \sum \sqrt{p\,q}$ (higher means more similar). The particle's weight is $w = \exp(20 \cdot \mathrm{BC})$, so better matches get much larger weights.
4. *Estimate.* We normalize the weights and report the weighted average of all particles, $E[s] = \sum_i W_i\, s_i$, as the tracked box.

What we changed from Exercise 3:

- *Automatic start.* Exercise 3 sets the first box by hand. We instead use the first frame where the mask is not empty and take its bounding box as the start, so the tracker starts on its own from the stage-2 mask. Frames before that just report this first box.
- *Reference from the person only.* Normally the reference histogram q is taken from the whole starting box. We take it only from the person's pixels (using the mask), so a box that mostly covers background cannot get a high score. This helps the tracker stay on the person.
- *Bigger noise.* Exercise 3 used small images with small noise. Our video is full-HD and the person moves about 10 px per frame and turns around, so we use larger noise ($\sigma_{\mathrm{pos}} = 10$, $\sigma_{\mathrm{vel}} = 3$ px). The exercise lets us choose these values.

The Bhattacharyya coefficient (how much two color histograms overlap) and the random resampling are both from the exercise/lecture. No library does the tracking for us – every step is written in NumPy, and we only use `cv2.rectangle` to draw the box. There is no deep learning and no built-in tracker, so the 70-point cap does not apply. With only 100 particles the stage stays well within the time limit.

Result. The green box follows the person for the whole clip, even when they turn around, and `tracking.json` has one `[ROW, COL, HEIGHT, WIDTH]` box per frame, numbered from "1".